

Lesson 12 - Assignment 2

Audit and Harden a Django Application

App Name:	Student Notes App
Student Name:	Syed Obaid
GitHub Repository Link:	https://github.com/syedobaidahmed33-lang/student_notes_security_audit_app

Project Summary

This package contains a complete Django Student Notes App prepared for a security and scalability audit. The application includes safe ORM queries, CSRF protection, template auto-escaping, environment-variable based settings, production security settings, caching, a SECURITY.md audit file, and a README scalability section.

Required Files Included

File	Purpose
README.md	Project overview, local setup, caching explanation, scalability plan
SECURITY.md	Every finding with risk, fix, and file location
.env.example	Required environment variables without real secret values
requirements.txt	Django and python-decouple dependencies
student_notes/settings.py	SECRET_KEY from environment, DEBUG, ALLOWED_HOSTS, HTTPS and cookie settings, cache configuration
notes/views.py	Safe ORM query, SQL injection comparison comment, cached view
templates/notes/*.html	POST forms with CSRF tokens and auto-escaped user data

Github repo:

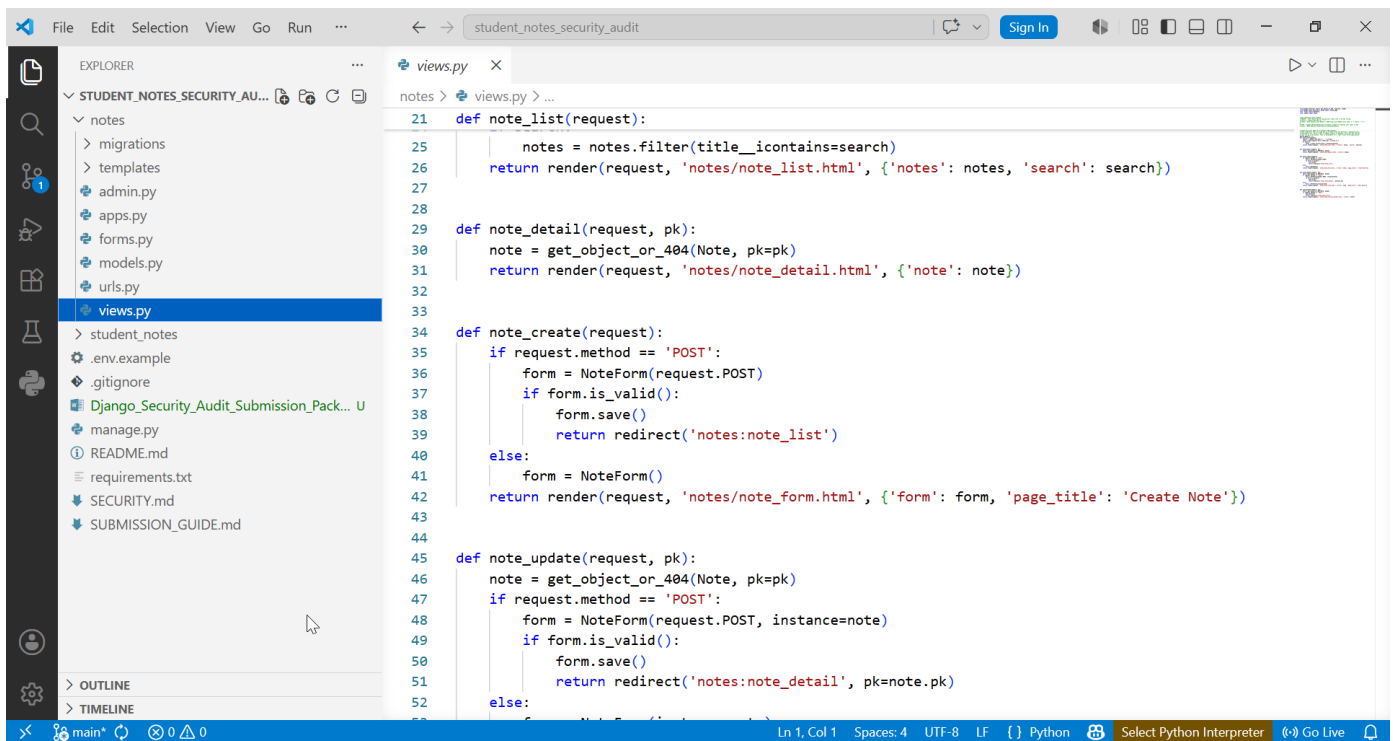
The screenshot shows a GitHub repository page for 'student_notes_security_audit_app' by user 'syedobaidahmed33-lang'. The repository is public and has 1 branch and 0 tags. The file list includes: notes, student_notes, .env.example, .gitignore, README.md, SECURITY.md, SUBMISSION_GUIDE.md, manage.py, and requirements.txt. The right sidebar shows the 'About' section with no description, a 'Readme' link, and 'Security policy' and 'Activity' links. The 'Releases' section shows no releases published, and the 'Packages' section shows no packages published. The 'Contributors' section shows 1 contributor.

Settings .py

The screenshot shows a code editor with the 'settings.py' file open. The file contains Django settings for the 'Student Notes App'. The settings include: BASE_DIR, SECRET_KEY (hardcoded for development), DEBUG (False), ALLOWED_HOSTS (localhost, 127.0.0.1), and INSTALLED_APPS (django.contrib.admin, django.contrib.auth, django.contrib.contenttypes, django.contrib.sessions, django.contrib.messages, django.contrib.staticfiles, notes).

```
1 """
2 Django settings for the Student Notes App.
3 This file was hardened for Lesson 12 - Assignment 2.
4 """
5
6 from pathlib import Path
7 from decouple import config, Csv
8
9 BASE_DIR = Path(__file__).resolve().parent.parent
10
11 # UNSAFE - never commit a real secret key to source control:
12 # SECRET_KEY = 'django-insecure-hardcoded-example-key'
13 # SAFE - read from environment so the secret does not appear in the codebase.
14 SECRET_KEY = config('DJANGO_SECRET_KEY', default='fallback-for-local-dev-only-change-me')
15
16 # In production, DJANGO_DEBUG should be False so error pages do not expose stack traces.
17 DEBUG = config('DJANGO_DEBUG', default=False, cast=bool)
18
19 # Only allow requests to known hostnames; this helps reduce Host header abuse.
20 ALLOWED_HOSTS = config('DJANGO_ALLOWED_HOSTS', default='localhost,127.0.0.1', cast=Csv())
21
22 INSTALLED_APPS = [
23     'django.contrib.admin',
24     'django.contrib.auth',
25     'django.contrib.contenttypes',
26     'django.contrib.sessions',
27     'django.contrib.messages',
28     'django.contrib.staticfiles',
29     'notes',
30 ]
```

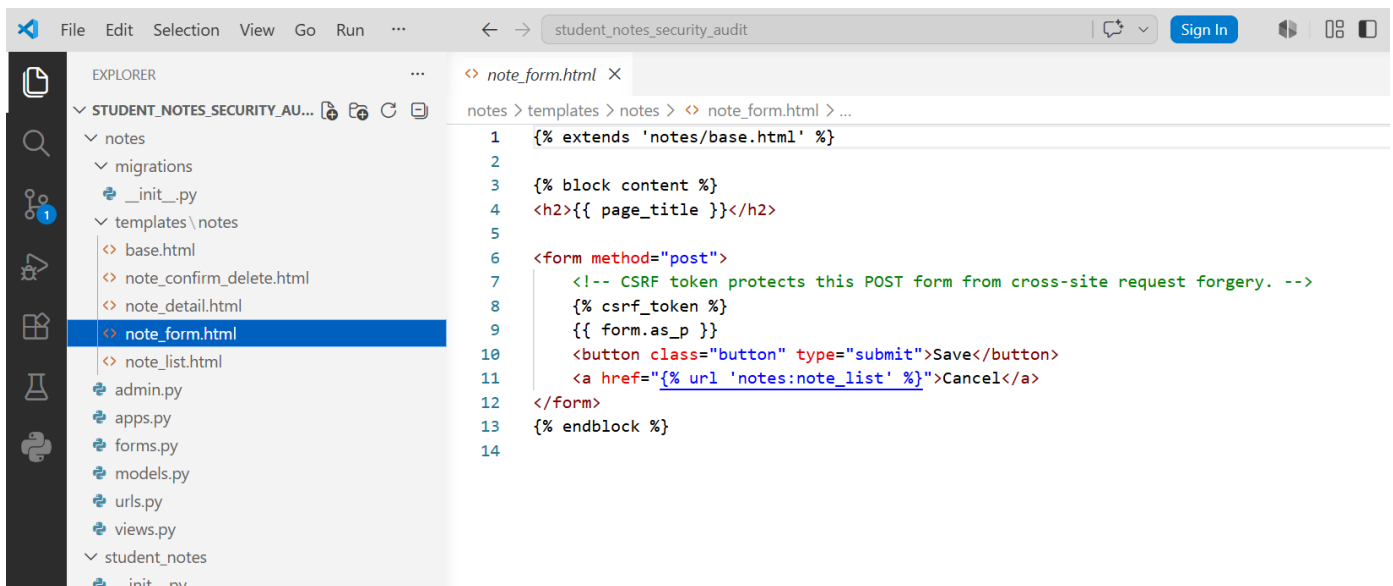
Views.py



The screenshot shows the Visual Studio Code editor with the Django project 'student_notes_security_audit' open. The Explorer sidebar on the left shows the project structure, with 'views.py' selected under the 'notes' directory. The main editor window displays the code for 'views.py', which includes functions for listing, creating, updating, and deleting notes. The code is as follows:

```
21 def note_list(request):
22     notes = notes.filter(title__icontains=search)
23     return render(request, 'notes/note_list.html', {'notes': notes, 'search': search})
24
25 def note_detail(request, pk):
26     note = get_object_or_404(Note, pk=pk)
27     return render(request, 'notes/note_detail.html', {'note': note})
28
29 def note_create(request):
30     if request.method == 'POST':
31         form = NoteForm(request.POST)
32         if form.is_valid():
33             form.save()
34             return redirect('notes:note_list')
35     else:
36         form = NoteForm()
37     return render(request, 'notes/note_form.html', {'form': form, 'page_title': 'Create Note'})
38
39 def note_update(request, pk):
40     note = get_object_or_404(Note, pk=pk)
41     if request.method == 'POST':
42         form = NoteForm(request.POST, instance=note)
43         if form.is_valid():
44             form.save()
45             return redirect('notes:note_detail', pk=note.pk)
46     else:
47         form = NoteForm(instance=note)
48     return render(request, 'notes/note_form.html', {'form': form, 'page_title': 'Update Note'})
```

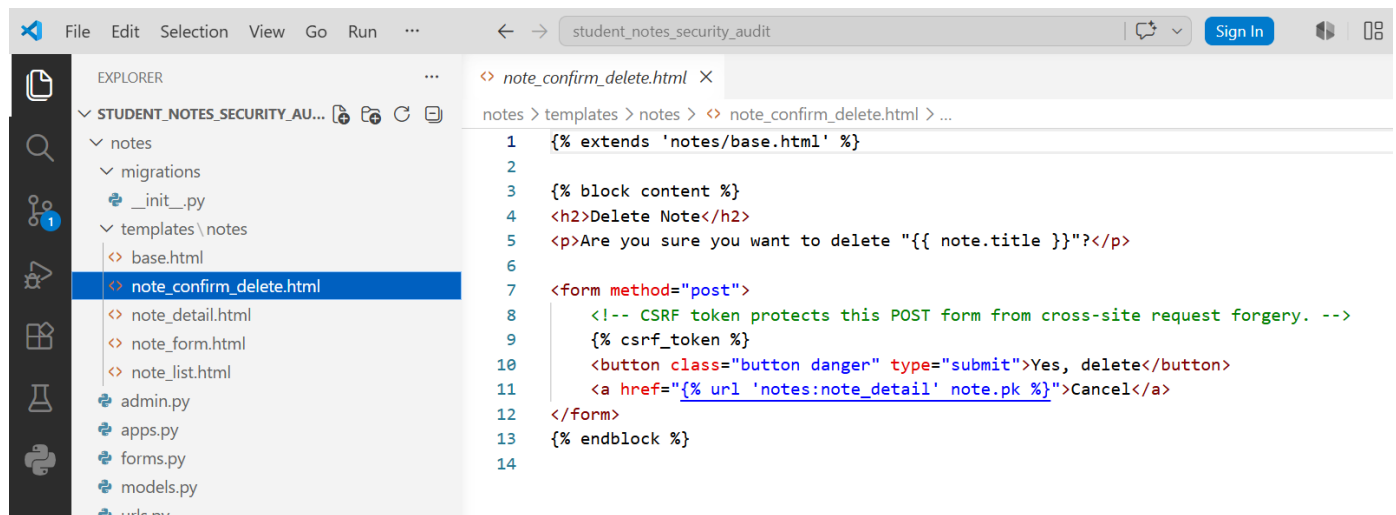
note_form.html



The screenshot shows the Visual Studio Code editor with the Django project 'student_notes_security_audit' open. The Explorer sidebar on the left shows the project structure, with 'note_form.html' selected under the 'templates/notes' directory. The main editor window displays the code for 'note_form.html', which is a Django template for creating and updating notes. The code is as follows:

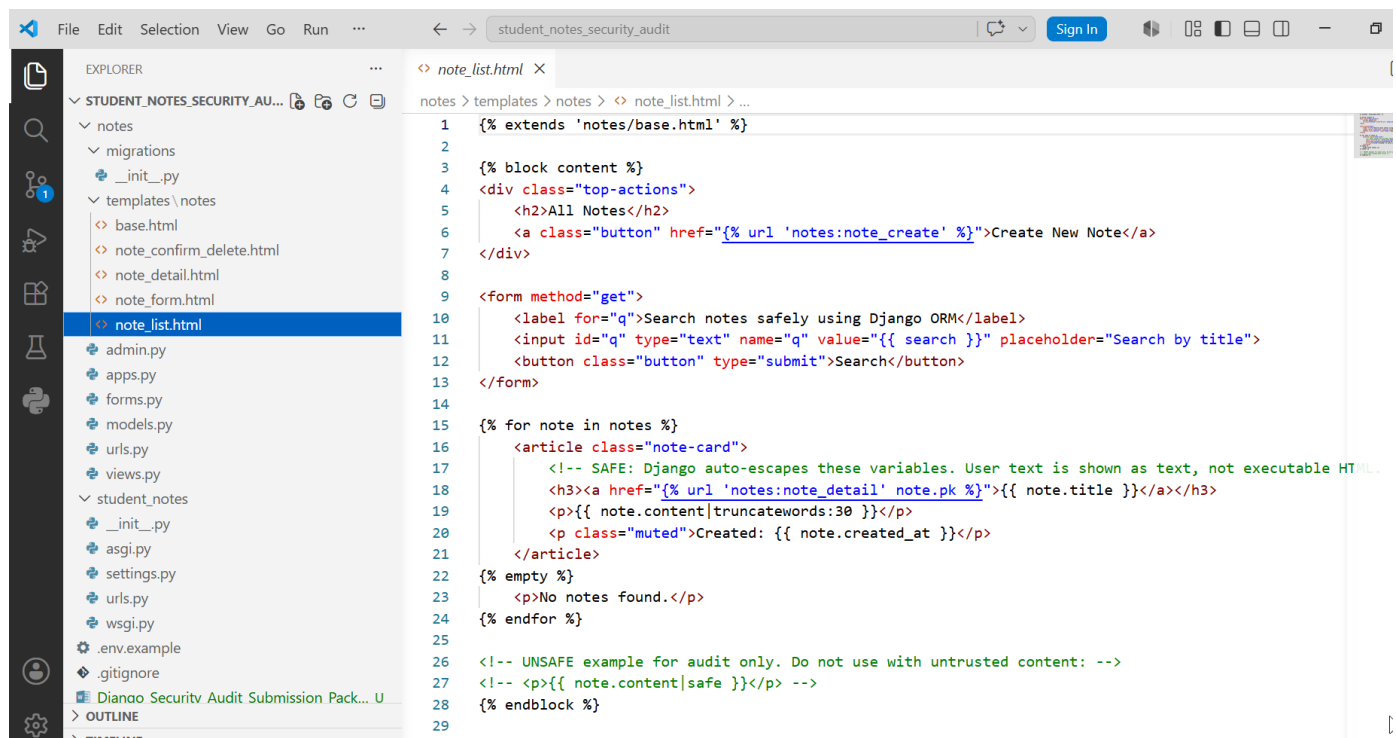
```
1 {% extends 'notes/base.html' %}
2
3 {% block content %}
4 <h2>{{ page_title }}</h2>
5
6 <form method="post">
7     <!-- CSRF token protects this POST form from cross-site request forgery. -->
8     {{ csrf_token }}
9     {{ form.as_p }}
10    <button class="button" type="submit">Save</button>
11    <a href="{% url 'notes:note_list' %}">Cancel</a>
12 </form>
13 {% endblock %}
14
```

note_confirm_delete.html



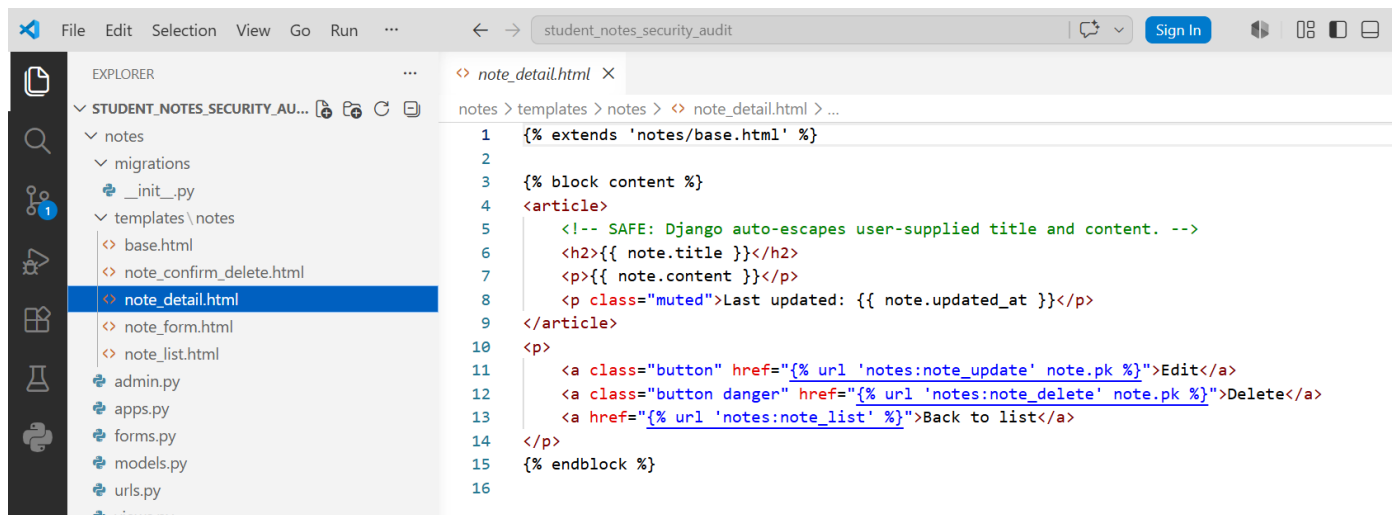
```
1 {% extends 'notes/base.html' %}
2
3 {% block content %}
4 <h2>Delete Note</h2>
5 <p>Are you sure you want to delete "{{ note.title }}"?</p>
6
7 <form method="post">
8     <!-- CSRF token protects this POST form from cross-site request forgery. -->
9     {% csrf_token %}
10    <button class="button danger" type="submit">Yes, delete</button>
11    <a href="{% url 'notes:note_detail' note.pk %}">Cancel</a>
12 </form>
13 {% endblock %}
14
```

note_list.html



```
1 {% extends 'notes/base.html' %}
2
3 {% block content %}
4 <div class="top-actions">
5     <h2>All Notes</h2>
6     <a class="button" href="{% url 'notes:note_create' %}">Create New Note</a>
7 </div>
8
9 <form method="get">
10     <label for="q">Search notes safely using Django ORM</label>
11     <input id="q" type="text" name="q" value="{{ search }}" placeholder="Search by title">
12     <button class="button" type="submit">Search</button>
13 </form>
14
15 {% for note in notes %}
16     <article class="note-card">
17         <!-- SAFE: Django auto-escapes these variables. User text is shown as text, not executable HTML. -->
18         <h3><a href="{% url 'notes:note_detail' note.pk %}">{{ note.title }}</a></h3>
19         <p>{{ note.content|truncatewords:30 }}</p>
20         <p class="muted">Created: {{ note.created_at }}</p>
21     </article>
22 {% empty %}
23     <p>No notes found.</p>
24 {% endfor %}
25
26 <!-- UNSAFE example for audit only. Do not use with untrusted content: -->
27 <!-- <p>{{ note.content|safe }}</p> -->
28 {% endblock %}
29
```

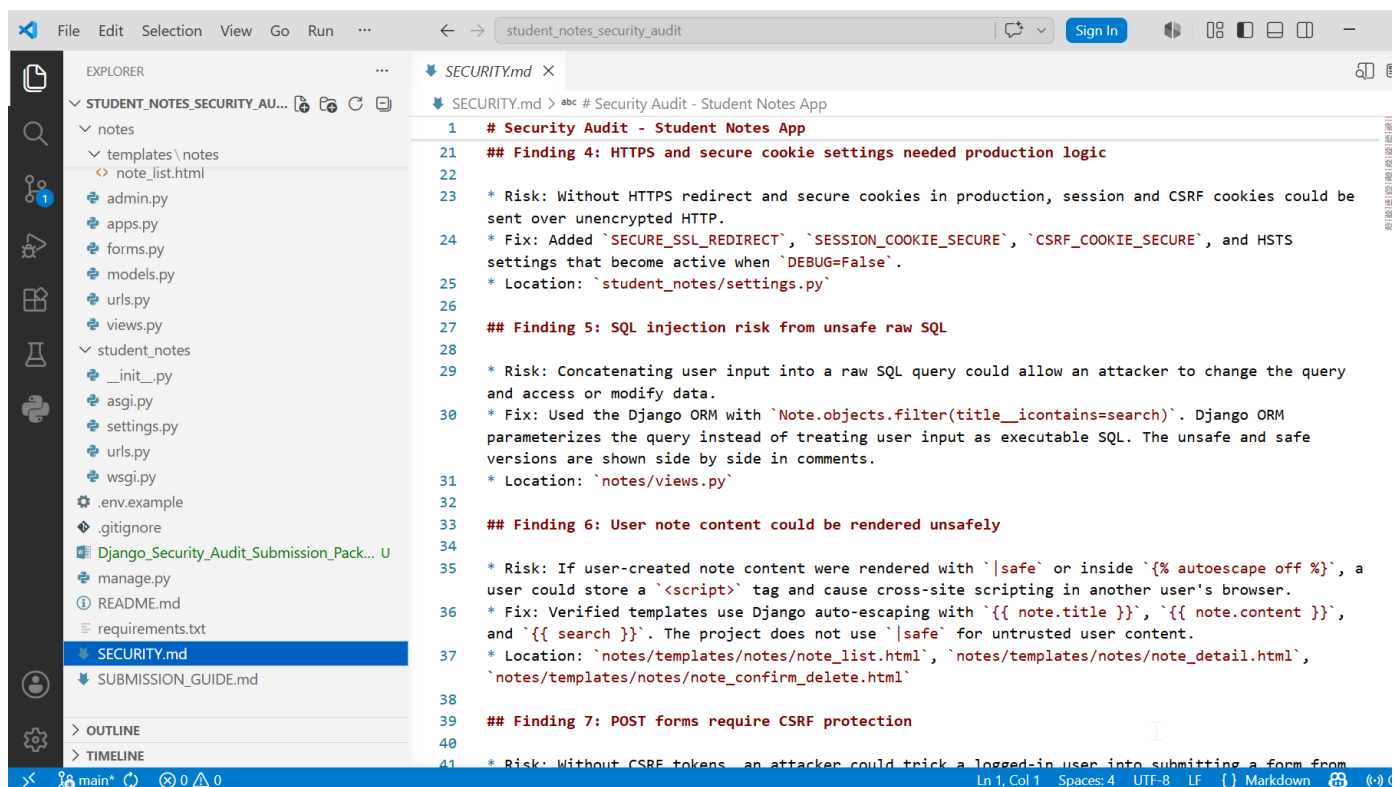
note_detail.html



The screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor on the right. The file explorer shows the project structure for 'STUDENT_NOTES_SECURITY_AUDIT', with the 'note_detail.html' file selected under the 'templates > notes' directory. The code editor displays the content of 'note_detail.html', which is a Django template that extends 'notes/base.html'. It contains a block of content with an article tag, a h2 tag for the note title, a p tag for the note content, and a p tag for the last updated time. There are also three links: 'Edit', 'Delete', and 'Back to list'.

```
1 {% extends 'notes/base.html' %}
2
3 {% block content %}
4 <article>
5     <!-- SAFE: Django auto-escapes user-supplied title and content. -->
6     <h2>{{ note.title }}</h2>
7     <p>{{ note.content }}</p>
8     <p class="muted">Last updated: {{ note.updated_at }}</p>
9 </article>
10 <p>
11     <a class="button" href="{% url 'notes:note_update' note.pk %}">Edit</a>
12     <a class="button danger" href="{% url 'notes:note_delete' note.pk %}">Delete</a>
13     <a href="{% url 'notes:note_list' %}">Back to list</a>
14 </p>
15 {% endblock %}
16
```

SECURITY.md



The screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor on the right. The file explorer shows the project structure for 'STUDENT_NOTES_SECURITY_AUDIT', with the 'SECURITY.md' file selected. The code editor displays the content of 'SECURITY.md', which is a Markdown file containing a security audit report. The report includes a title, a finding about HTTP and secure cookie settings, a finding about SQL injection risk, a finding about user note content rendering, and a finding about POST forms requiring CSRF protection.

```
1 # Security Audit - Student Notes App
2
3 ## Finding 4: HTTP and secure cookie settings needed production logic
4
5 * Risk: Without HTTPS redirect and secure cookies in production, session and CSRF cookies could be sent over unencrypted HTTP.
6
7 * Fix: Added `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, and HSTS settings that become active when `DEBUG=False`.
8
9 * Location: `student_notes/settings.py`
10
11 ## Finding 5: SQL injection risk from unsafe raw SQL
12
13 * Risk: Concatenating user input into a raw SQL query could allow an attacker to change the query and access or modify data.
14
15 * Fix: Used the Django ORM with `Note.objects.filter(title__icontains=search)`. Django ORM parameterizes the query instead of treating user input as executable SQL. The unsafe and safe versions are shown side by side in comments.
16
17 * Location: `notes/views.py`
18
19 ## Finding 6: User note content could be rendered unsafely
20
21 * Risk: If user-created note content were rendered with `|safe` or inside `{% autoescape off %}`, a user could store a `
```

Security Fix Evidence

1. SQL Injection Prevention

The app avoids raw SQL string concatenation and uses Django ORM filters for user search input.

```
# UNSAFE - never build SQL by joining user input into a raw SQL string.
# notes = Note.objects.raw("SELECT * FROM notes_note WHERE title LIKE '%" + search + "%' ")
# SAFE - Django ORM parameterizes the query instead of treating input as SQL.
notes = Note.objects.filter(title__icontains=search)
```

2. CSRF Protection

Every template form that submits a POST request includes {% csrf_token %}.

```
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Save</button>
</form>
```

3. XSS Prevention

Templates use normal Django variables, which are auto-escaped by default. The project does not use the safe filter for untrusted note content.

```
<h3>{{ note.title }}</h3>
<p>{{ note.content }}</p>
<!-- UNSAFE example only: {{ note.content|safe }} -->
```

Environment Variables and Production Settings

The SECRET_KEY and deployment settings are controlled through environment variables. Real secrets should never be committed to GitHub.

```
SECRET_KEY = config('DJANGO_SECRET_KEY', default='fallback-for-local-dev-only-change-me')
DEBUG = config('DJANGO_DEBUG', default=False, cast=bool)
ALLOWED_HOSTS = config('DJANGO_ALLOWED_HOSTS', default='localhost,127.0.0.1', cast=Csv())
SECURE_SSL_REDIRECT = not DEBUG
SESSION_COOKIE_SECURE = not DEBUG
CSRF_COOKIE_SECURE = not DEBUG
```

Caching Evidence

The notes list is cached for 5 minutes. This reduces repeated database work for a commonly visited page.

```
# Cache this list page for 5 minutes (300 seconds).
# Stale-data risk: users may see the old list for up to 5 minutes.
@cache_page(60 * 5)
def note_list(request):
    notes = Note.objects.all().order_by('-created_at')
```

Scalability Plan

If this Django application needed to support 10x more users, I would first use vertical scaling for a quick short-term improvement by increasing CPU and memory on the server. After that, I would move to horizontal scaling by running multiple Django application servers behind a load balancer. The load balancer would distribute incoming requests across the servers so no single server becomes overloaded. Because users may be sent to different servers on different requests, sessions should be stored in a shared location such as database-backed sessions or Redis instead of local server memory. For the database, I would start with read replication if many users are reading notes at the same time. Partitioning would only be needed later if the database tables became very large.

How to Run the Project

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
python manage.py makemigrations
python manage.py migrate
python manage.py runserver
```

Open the browser at: <http://127.0.0.1:8000/>

GitHub Upload Commands

```
git init
git add .
git commit -m "Add hardened Django student notes app"
git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

Important: Do not upload .env or db.sqlite3. The included .gitignore file blocks them.